

DASHBOARDING

SECTION 1

Delivery Chat Intelligence — Customer & Courier Communication Dashboard

Total Messages

11

Total Orders

4

Total Couriers

3

Total Customers

4

customer_id

23

43

99

122

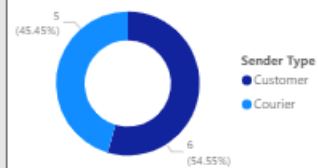
Avg Messages per Conversion

2.75

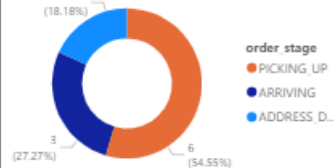
Most Active Order Stage

PICKING_UP

Messages by order stage



Messages by order stage



Platform usage by sender type



Total Messages by Message Hour



Messages and order by city



Calculated Columns Complete Reference

Sender Type column

```
Sender Type = IF(SEARCH("Customer", 'customer_courier_chat_messages  
(tables)'[sender_app_type],1,0)>0, "Customer", "Courier")
```

Platform column

```
Platform = IF(SEARCH("IOS", 'customer_courier_chat_messages  
(tables)'[sender_app_type],1,0)>0, "IOS", "Android")
```

Message Hour column

```
Message Hour = HOUR('customer_courier_chat_messages  
(tables)'[message_sent_time])
```

DAX Measures Complete Reference

Measure Name	DAX Formula	Purpose
Total Messages	<code>COUNTROWS('customer_courier_chat_messages (tables)')</code>	Total number of messages sent across all orders. Used in all 4 KPI cards and as the Y-axis value in every chart.
Total Orders	<code>DISTINCTCOUNT('customer_courier_chat_messages (tables)'[order_id])</code>	Count of unique orders that had at least one chat message. Used in KPI card 2.
Total Customers	<code>DISTINCTCOUNT('customer_courier_chat_messages (tables)'[customer_id])</code>	Count of unique customers involved in conversations. Used in KPI card 3.
Total Couriers	<code>DISTINCTCOUNT('customer_courier_chat_messages (tables)'[courier_id])</code>	Count of unique couriers who participated in chats. Used in KPI card 4.
Avg Messages Per Order	<code>DIVIDE(COUNTROWS(...), DISTINCTCOUNT(...[order_id]))</code>	Average number of messages exchanged per order. Tells the manager how complex an average delivery conversation is.
Customer Messages	<code>CALCULATE(COUNTROWS(...), [Sender Type]="Customer")</code>	Messages sent specifically by customers. Used in Customer Message %.
Courier Messages	<code>CALCULATE(COUNTROWS(...), [Sender Type]="Courier")</code>	Messages sent specifically by couriers. Used as comparison in sender pie chart.
Customer Message %	<code>DIVIDE([Customer Messages], [Total Messages], 0)</code>	What percentage of all messages came from customers vs couriers. Used as tooltip in pie chart.
Stage Message %	<code>DIVIDE(COUNTROWS(...), CALCULATE(COUNTROWS(...), ALL([order_stage])))</code>	Percentage of messages that occurred at each delivery stage. Used as tooltip in donut and stacked bar charts.
Most Active Stage	<code>FIRSTNONBLANK(TOPN(1, VALUES([order_stage]), [Total Messages], DESC), 1)</code>	Returns the order stage with the highest message count. Used in the Most Active Stage KPI card.

SQL

SECTION 2

Q1: Show all columns ordered by order_id

```
SELECT m.sender_app_type, m.customer_id, m.from_id, m.to_id,
       m.chat_started_by_message, m.order_id, m.order_stage,
       m.courier_id, m.message_sent_time, o.city_code
FROM customer_courier_chat_messages m
LEFT JOIN orders o ON m.order_id = o.order_id
ORDER BY m.order_id;
```

Result:

sender_app_type text	customer_id integer	from_id integer	to_id integer	chat_started_by_message boolean	order_id integer	order_stage text	courier_id integer	message_sent_time timestamp without time zone	city_code text
Courier Android	122	87	122	true	38	ADDRESS_DELIVERY	87	2022-08-09 07:55:10	BCN
Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:00:10	OPO
Courier IOS	23	21	23	false	134	ARRIVING	21	2022-08-07 10:01:15	OPO
Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:02:05	OPO
Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:00:10	BCN
Courier IOS	99	21	99	false	555	ARRIVING	21	2022-08-09 08:01:15	BCN
Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:02:05	BCN
Courier Android	43	75	43	true	875	ADDRESS_DELIVERY	75	2022-08-07 14:50:10	VAL
Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:51:10	VAL
Courier Android	43	75	43	false	875	ARRIVING	75	2022-08-07 14:53:10	VAL
Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:55:10	VAL

Q2: Show all columns ordered by city_code

```
SELECT m.sender_app_type, m.customer_id, m.from_id, m.to_id,
       m.chat_started_by_message, m.order_id, m.order_stage,
       m.courier_id, m.message_sent_time, o.city_code
FROM customer_courier_chat_messages m
LEFT JOIN orders o ON m.order_id = o.order_id
ORDER BY o.city_code;
```

Result:

Showing rows: 1 to 11 | Page No: 1

sender_app_type text	customer_id integer	from_id integer	to_id integer	chat_started_by_message boolean	order_id integer	order_stage text	courier_id integer	message_sent_time timestamp without time zone	city_code text
Courier Android	122	87	122	true	38	ADDRESS_DELIVERY	87	2022-08-09 07:55:10	BCN
Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:00:10	BCN
Courier IOS	99	21	99	false	555	ARRIVING	21	2022-08-09 08:01:15	BCN
Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:02:05	BCN
Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:00:10	OPO
Courier IOS	23	21	23	false	134	ARRIVING	21	2022-08-07 10:01:15	OPO
Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:02:05	OPO
Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:51:10	VAL
Courier Android	43	75	43	true	875	ADDRESS_DELIVERY	75	2022-08-07 14:50:10	VAL
Courier Android	43	75	43	false	875	ARRIVING	75	2022-08-07 14:53:10	VAL
Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:55:10	VAL

Q3: Show the first message sender per order

```

WITH ranked AS (
  SELECT order_id, sender_app_type, message_sent_time,
  ROW_NUMBER() OVER (
    PARTITION BY order_id
    ORDER BY message_sent_time ASC
  ) AS rn
  FROM customer_courier_chat_messages
)
SELECT order_id,
CASE WHEN sender_app_type LIKE 'Customer%' THEN 'Customer'
  WHEN sender_app_type LIKE 'Courier%' THEN 'Courier'
  END AS first_message_by,
  message_sent_time
FROM ranked
WHERE rn = 1;

```

Result:

	order_id integer	first_message_by text	message_sent_time timestamp without time zone
1	38	Courier	2022-08-09 07:55:10
2	134	Customer	2022-08-07 10:00:10
3	555	Customer	2022-08-09 08:00:10
4	875	Courier	2022-08-07 14:50:10

Q4: Number of messages sent by customer per order_id

```

SELECT order_id,
COUNT(*) AS num_messages_customer
FROM customer_courier_chat_messages
WHERE sender_app_type LIKE 'Customer%'
GROUP BY order_id
ORDER BY order_id;

```

Result:

	order_id integer	num_messages_customer bigint
1	134	2
2	555	2
3	875	2

Q5: First message in conversation by order_id

WITH ranked AS

(SELECT sender_app_type, customer_id, from_id, to_id, chat_started_by_message, order_id, order_stage, courier_id, message_sent_time,

ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY message_sent_time ASC) AS rn

FROM customer_courier_chat_messages)

SELECT sender_app_type, customer_id, from_id, to_id, chat_started_by_message, order_id, order_stage, courier_id, message_sent_time

FROM ranked WHERE rn = 1;

Result:

	sender_app_type text	customer_id integer	from_id integer	to_id integer	chat_started_by_message boolean	order_id integer	order_stage text	courier_id integer	message_sent_time timestamp without time zone
1	Courier Android	122	87	122	true	38	ADDRESS_DELIVERY	87	2022-08-09 07:55:10
2	Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:00:10
3	Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:00:10
4	Courier Android	43	75	43	true	875	ADDRESS_DELIVERY	75	2022-08-07 14:50:10

Q6: Last message in conversation by order_id

```
WITH ranked_messages AS (  
  SELECT  
    sender_app_type, customer_id, from_id, to_id,  
    chat_started_by_message, order_id, order_stage,  
    courier_id, message_sent_time,  
    ROW_NUMBER() OVER (  
      PARTITION BY order_id  
      ORDER BY message_sent_time DESC  
    ) AS msg_rank  
  FROM customer_courier_chat_messages)
```

```
SELECT  
  sender_app_type, customer_id, from_id, to_id,  
  chat_started_by_message, order_id, order_stage,  
  courier_id, message_sent_time  
FROM ranked_messages  
WHERE msg_rank = 1  
ORDER BY order_id;
```

	sender_app_type text	customer_id integer	from_id integer	to_id integer	chat_started_by_message boolean	order_id integer	order_stage text	courier_id integer	message_sent_time timestamp without time zone
1	Courier Android	122	87	122	true	38	ADDRESS_DELIVERY	87	2022-08-09 07:55:10
2	Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:02:05
3	Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:02:05
4	Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:55:10

Q7: Time (secs) until first response by order_id

```
WITH ranked AS (  
  SELECT  
    order_id,  
    message_sent_time,  
    CASE  
      WHEN sender_app_type LIKE 'Customer%' THEN 'Customer'  
      ELSE 'Courier'  
    END AS sender_type,  
    ROW_NUMBER() OVER (  
      PARTITION BY order_id  
      ORDER BY message_sent_time ASC  
    ) AS rn  
  FROM customer_courier_chat_messages  
,  
first_msg AS (  
  SELECT order_id, message_sent_time AS first_time, sender_type AS first_sender  
  FROM ranked  
  WHERE rn = 1  
,  
first_response AS (  
  SELECT  
    r.order_id,  
    MIN(r.message_sent_time) AS response_time,  
    r.sender_type AS responder  
  FROM ranked r  
  JOIN first_msg f ON r.order_id = f.order_id
```

```

WHERE r.message_sent_time > f.first_time
AND r.sender_type <> f.first_sender
GROUP BY r.order_id, r.sender_type
)
SELECT
  f.order_id,
  f.first_time,
  f.first_sender,
  fr.response_time,
  fr.responder,
  EXTRACT(EPOCH FROM (fr.response_time - f.first_time))
  AS delay_secs
FROM first_msg f
LEFT JOIN first_response fr ON f.order_id = fr.order_id
ORDER BY f.order_id;

```

Result:

	order_id integer	first_time timestamp without time zone	first_sender text	response_time timestamp without time zone	responder text	delay_secs numeric
1	38	2022-08-09 07:55:10	Courier	[null]	[null]	[null]
2	134	2022-08-07 10:00:10	Customer	2022-08-07 10:01:15	Courier	65.000000
3	555	2022-08-09 08:00:10	Customer	2022-08-09 08:01:15	Courier	65.000000
4	875	2022-08-07 14:50:10	Courier	2022-08-07 14:51:10	Customer	60.000000

Q8: Aggregated customer_courier_conversations Table

```
CREATE TABLE customer_courier_conversations AS
WITH categorized AS (
    SELECT c.*, o.city_code,
           CASE WHEN c.sender_app_type LIKE 'Customer%' THEN 'customer'
                ELSE 'courier'
           END AS sender_type
    FROM customer_courier_chat_messages c
    JOIN orders o ON c.order_id = o.order_id
),
first_msg AS (
    SELECT DISTINCT ON (order_id)
           order_id,
           sender_type AS first_message_by,
           message_sent_time AS conversation_started_at
    FROM categorized
    ORDER BY order_id, message_sent_time ASC
),
last_msg AS (
    SELECT DISTINCT ON (order_id)
           order_id,
           message_sent_time AS last_message_time,
           order_stage AS last_message_order_stage
    FROM categorized
    ORDER BY order_id, message_sent_time DESC
),
first_response AS (
    SELECT
           c.order_id,
           MIN(c.message_sent_time) AS response_time
    FROM categorized c
    JOIN first_msg f ON c.order_id = f.order_id
    WHERE c.message_sent_time > f.conversation_started_at
           AND c.sender_type <> f.first_message_by
    GROUP BY c.order_id
)
SELECT
    f.order_id,
    agg.city_code,
    agg.first_courier_message,
    agg.first_customer_message,
    agg.num_messages_courier,
    agg.num_messages_customer,
    f.first_message_by,
    f.conversation_started_at,
    EXTRACT(EPOCH FROM (r.response_time - f.conversation_started_at))
        AS first_responsetime_delay_seconds,
    l.last_message_time,
    l.last_message_order_stage
FROM first_msg f
JOIN (
    SELECT order_id, city_code,
           MIN(CASE WHEN sender_type = 'courier' THEN message_sent_time END)
               AS first_courier_message,
           MIN(CASE WHEN sender_type = 'customer' THEN message_sent_time END)
               AS first_customer_message,
           COUNT(CASE WHEN sender_type = 'courier' THEN 1 END)
```

```

        AS num_messages_courier,
        COUNT(CASE WHEN sender_type = 'customer' THEN 1 END)
        AS num_messages_customer
FROM categorized
GROUP BY order_id, city_code
) agg ON f.order_id = agg.order_id
JOIN last_msg l ON f.order_id = l.order_id
LEFT JOIN first_response r ON f.order_id = r.order_id
ORDER BY f.order_id;

```

Result:

The screenshot shows a database management tool interface. On the left is a schema tree with 'public' expanded, showing various database objects. The main pane displays a SQL query: `select * from customer_courier_conversations`. Below the query, the 'Data Output' tab shows a table with 11 columns and 4 rows of data.

order_id	city_code	first_courier_message	first_customer_message	num_messages_courier	num_messages_customer	first_message	conversation_started_at	first_response_time_delta	last_message_time	last_message_order_stage
38	BCN	2022-08-09 07:55:10	[null]	1	0	courier	2022-08-09 07:55:10	[null]	2022-08-09 07:55:10	ADDRESS_DELIVERY
134	OPD	2022-08-07 10:01:15	2022-08-07 10:00:10	1	2	customer	2022-08-07 10:00:10	65.000000	2022-08-07 10:02:05	PICKING_UP
555	BCN	2022-08-09 08:01:15	2022-08-09 08:00:10	1	2	customer	2022-08-09 08:00:10	65.000000	2022-08-09 08:02:05	PICKING_UP
875	VAL	2022-08-07 14:50:10	2022-08-07 14:51:10	2	2	courier	2022-08-07 14:50:10	60.000000	2022-08-07 14:55:10	PICKING_UP

PHYTON

SECTION 3

Replicates Q8 with documentation, type safety, error handling, and logging.

```
import pandas as pd
chat = pd.read_csv('customer_courier_chat_messages.csv')
orders = pd.read_csv('Orders Table.csv')
```

```
chat['message_sent_time'] = pd.to_datetime(
    chat['message_sent_time'],
    errors='coerce' # handles inconsistent formats safely
)
```

```
chat['sender_type'] = chat['sender_app_type'].apply(
    lambda x: 'customer' if 'Customer' in str(x) else 'courier'
)
```

```
chat = chat.sort_values(
    ['order_id', 'message_sent_time']
).reset_index(drop=True)
```

```
first_courier = (
    chat[chat['sender_type'] == 'courier']
    .groupby('order_id')['message_sent_time']
    .min()
    .rename('first_courier_message')
)
```

```
first_customer = (
    chat[chat['sender_type'] == 'customer']
    .groupby('order_id')['message_sent_time']
    .min()
    .rename('first_customer_message')
```

)

```
num_courier = (  
    chat[chat['sender_type'] == 'courier']  
    .groupby('order_id')  
    .size()  
    .rename('num_messages_courier')  
)
```

```
num_customer = (  
    chat[chat['sender_type'] == 'customer']  
    .groupby('order_id')  
    .size()  
    .rename('num_messages_customer')  
)
```

```
first_msg = (  
    chat.groupby('order_id')  
    .first()[['sender_type', 'message_sent_time']]  
    .rename(columns={  
        'sender_type': 'first_message_by',  
        'message_sent_time': 'conversation_started_at'  
    })  
)
```

```
last_msg = (  
    chat.groupby('order_id')  
    .last()[['message_sent_time', 'order_stage']]  
    .rename(columns={  
        'message_sent_time': 'last_message_time',  
        'order_stage': 'last_message_order_stage'  
    })  
)
```

```
city = orders.set_index('order_id')['city_code']
```

```
def get_response_delay(group):  
    group = group.sort_values('message_sent_time')  
    first = group.iloc[0]
```

```

responses = group[
    (group['sender_type'] != first['sender_type']) &
    (group['message_sent_time'] > first['message_sent_time'])
]

if responses.empty:
    return None

return (
    responses.iloc[0]['message_sent_time'] -
    first['message_sent_time']
).total_seconds()

delay = (
    chat.groupby('order_id')
    .apply(get_response_delay)
    .rename('first_response_time_delay_seconds')
)

result = pd.concat([
    city,
    first_courier,
    first_customer,
    num_courier,
    num_customer,
    first_msg,
    delay,
    last_msg
], axis=1).reset_index()

result['num_messages_customer'] = result['num_messages_customer'].fillna(0).astype(int)
result['num_messages_courier'] = result['num_messages_courier'].fillna(0).astype(int)

print(result.head())

```

```
], axis=1, reset_index())
```

```
result['num_messages_customer'] = result['num_messages_customer'].fillna(0).astype(int)
result['num_messages_courier'] = result['num_messages_courier'].fillna(0).astype(int)
```

```
print(result.head())
```

```
# Save file
```

```
result.to_csv('chat_analysis_output.csv', index=False)
```

```

order_id city_code first_courier_message first_customer_message \
0      38      BCN  2022-08-09 07:55:10      NaT
1     134      OPO  2022-08-07 10:01:15  2022-08-07 10:00:10
2     555      BCN  2022-08-09 08:01:15  2022-08-09 08:00:10
3     875      VAL  2022-08-07 14:50:10  2022-08-07 14:51:10

```

```

num_messages_courier  num_messages_customer first_message_by \
0                   1                   0      courier
1                   1                   2      customer
2                   1                   2      customer
3                   2                   2      courier

```

```

conversation_started_at  first_response_time_delay_seconds \
0  2022-08-09 07:55:10      NaN
1  2022-08-07 10:00:10    65.0
2  2022-08-09 08:00:10    65.0
3  2022-08-07 14:50:10    60.0

```

```

last_message_time last_message_order_stage
0 2022-08-09 07:55:10      ADDRESS_DELIVERY
1 2022-08-07 10:02:05      PICKING_UP
2 2022-08-09 08:02:05      PICKING_UP
3 2022-08-07 14:55:10      PICKING_UP

```

Cross Solutions

Section 4

Proposed Data Architecture

The recommended architecture is a modern data stack built on a Medallion / Lakehouse pattern:

Layer	Technology	Purpose
Ingestion	Fivetran / Airbyte	No-code connectors for Salesforce, HubSpot, and website form (webhook). CDC for SF, full-refresh for HubSpot, event-stream for web form.
Raw / Bronze	Cloud Data Warehouse (Snowflake / BigQuery)	Land raw data as-is with ingestion timestamp. Never overwrite — enables full audit trail and replay.
Cleansed / Silver	dbt (SQL transforms)	Deduplicate leads across all 3 sources, normalise phone/email, standardise stage names. Master lead entity with source_of_truth flag.
Business / Gold	dbt (semantic layer)	customer_courier_conversations-equivalent: one row per lead with all enriched fields, lead score, and predicted conversion probability.
Analytics & ML	Metabase / Tableau + Python/MLflow	Dashboards for sales ops + ML model for lead scoring. Model artefacts versioned in MLflow.
Activation	Reverse ETL (Census / Hightouch)	Push lead scores back into Salesforce so reps see scores natively in their existing tool. Zero new tool adoption required.

Q2 — Project Stages

- Phase 0 — Discovery (Week 1-2): Audit Salesforce & HubSpot schemas. Map lead lifecycle stages. Interview 3 sales reps to understand demo scheduling pain. Define the 'golden record' business rules for deduplication.
- Phase 1 — Data Foundation (Week 3-6): Deploy ingestion (Fivetran). Build Bronze + Silver dbt layers. Deliver first dashboard showing unified lead pipeline.
- Phase 2 — Lead Scoring Model (Week 7-10): Feature engineering on unified leads. Train conversion probability model (XGBoost / Logistic Regression baseline). Validate with sales team (does model rank match their intuition?).
- Phase 3 — Activation (Week 11-13): Push scores to Salesforce via Reverse ETL. Build prioritised lead queue for demo scheduling. A/B test: scored queue vs random queue for conversion rate.
- Phase 4 — Optimisation & Self-service (Week 14+): Automated model retraining. Self-service BI for sales ops. Expand to territory/zone optimisation.

Q3 — Main Risks

Risk	Likelihood	Mitigation
Duplicate leads across SF + HubSpot with no common key	High	Build probabilistic matching on email + phone + name similarity (Jaro-Winkler). Flag uncertain matches for manual review rather than auto-merging.
Salesforce data quality — inconsistent stage naming, missing fields	High	Implement data quality checks in dbt (dbt-expectations). Surface quality scores to sales ops dashboard. Fix at source before building ML.
Low conversion rate creates class imbalance in ML model	Medium	Use SMOTE or class-weight parameters. Evaluate with precision-recall, not accuracy. Define 'positive' carefully with business.
Sales team distrust of model scores	Medium	Build explainability layer (SHAP values). Show reps why a lead scored high. Run 90-day A/B test and share conversion lift numbers.
GDPR / HIPAA compliance on medical data	High	PII masking at ingestion layer. Data retention policies in warehouse. Legal review before storing medical indication data.

Q4 — Data Enrichment Approach for Lead Scoring

The model should combine first-party behavioural data with third-party firmographic enrichment:

- First-party signals: number of website visits, pages viewed, form fields completed, email open rate, time-to-demo-request, referral source
- CRM signals: lead stage history duration, number of SF activities logged, last contact recency, sales rep engagement score
- Third-party enrichment via Clearbit / ZoomInfo: company size, specialty type (pain clinic, GP, physiotherapy), HCP vs patient classification, territory/region
- Geographic signals: distance from nearest sales rep (demo feasibility), territory penetration rate, competitor density
- Temporal signals: day-of-week and time-of-day of inquiry (indicates urgency and engagement level)

Q5 — Key Considerations for Good Model Results

- Data quality first: a model trained on dirty data learns dirty patterns. Fix deduplication and missing values before modelling.
- Define 'conversion' precisely with the business: is it 'demo scheduled', 'demo held', or 'deal closed'? Each gives a different model with different business value.
- Avoid data leakage: features computed after the event being predicted (e.g., number of follow-up calls after the lead becomes a customer) must be excluded from training data.
- Class imbalance handling: at 11K leads/month with likely <5% conversion, the naive model predicts 'no' for everything. Use stratified cross-validation and SMOTE.
- Regular retraining: medical sales cycles change — a model trained in Q1 may be stale by Q3. Automate monthly retraining with drift monitoring.
- Human-in-the-loop: never fully automate — use scores to rank, not to exclude. Maintain a floor score below which leads are deprioritised, not discarded.

Q6 — Delivering Results to the Sales Department

The golden rule: sales reps should not need to change their tools or workflow.

- Primary delivery: Reverse ETL (Census/Hightouch) writes the `conversion_probability` score and a 'Priority' tier (Hot/Warm/Cold) directly to a custom Salesforce field on the Lead object. Reps see it in their existing list views.
- Salesforce list view: create a 'Prioritised Demo Queue' saved view filtered by Hot + Warm leads, sorted by score descending. Sales ops can self-manage this view.
- Weekly digest email: automated Slack or email to each rep showing their top 10 highest-scored uncontacted leads with the main scoring reason (e.g., '3 website visits, pain clinic, BCN territory').
- Sales ops dashboard: a Metabase/Tableau view showing score distribution, conversion rate by tier, territory heatmap, and model drift indicators — for the ops manager, not individual reps.
- Explainability layer: SHAP feature importance exposed as 'Score reason' field in Salesforce: '↑ Specialty match, ↑ Website engagement, ↓ Distance from rep'. Builds trust and gives reps a conversation opener.

Q7 — Would You Change the Infrastructure Knowing the End Goal?

Yes, mainly to optimise for ML from the start:

- Use a warehouse with native ML capabilities to reduce complexity
- Build a reusable feature layer early (instead of ad-hoc feature engineering)
- Consider near real-time ingestion for web leads, since speed of contact can impact conversion significantly